课外思考题四

2025 年 10 月 19 日

T1：顺时针打印二维矩阵

1 问题分析

题目要求：

给定一个二维矩阵（m×n），从**最外层到最内层**，按**顺时针方向**输出矩阵中的所有元素。

例子：

矩阵

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

顺时针输出

输出顺序：1, 2, 3, 4, 8, 12, 16, 15, 14, 13, 9, 5, 6, 7, 11, 10

2 算法思路

定义四个边界指针：

top = 0
bottom = m - 1
left = 0
right = n - 1

在每一轮循环中依次执行：

1. 从左到右输出上边；
2. 从上到下输出右边；
3. 从右到左输出下边；
4. 从下到上输出左边；

然后缩小边界范围：

top++， bottom--， left++， right—

直到所有层都输出。

3 C 语言实现代码

```c
#include <stdio.h>
#include <stdlib.h>
void printMatrix(int** arr, int m, int n) {
    int top = 0; int bottom = m - 1;
    int left = 0; int right = n - 1;
    while (top <= bottom && left <= right) {
        for (int i = left; i <= right; i++)
            printf("%d ", arr[top][i]);
        top++;
        for (int i = top; i <= bottom; i++)
            printf("%d ", arr[i][right]);
        right--;
        if (top <= bottom) {
            for (int i = right; i >= left; i--)
                printf("%d ", arr[bottom][i]);
            bottom--;
        }
        if (left <= right) {
            for (int i = bottom; i >= top; i--)
                printf("%d ", arr[i][left]);
            left++;
        }
    }
}
int main(void) {
    int m, n;
    printf("输入行数m与列数n\n");
    scanf("%d%d", &m, &n);
    int** arr = (int**)malloc(m * sizeof(int*));
    for (int i = 0; i < m; i++) {
        arr[i] = (int*)malloc(sizeof(int) * n);
    }
    printf("输入矩阵元素：\n");
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &arr[i][j]);
        }
    }
    printMatrix(arr, m, n);
    for (int i = 0; i < m; i++) {
        free(arr[i]);
    }
    free(arr);
    return 0;
}
```

4 运行示例

```
输入行数m与列数n
4
4
输入矩阵元素：
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
1 2 3 4 8 12 16 15 14 13 9 5 6 7 11 10
```

5 性能分析

项目	复杂度
时间复杂度	O(m×n)： 每个元素访问一次
空间复杂度	O(1)： 仅使用常量级辅助变量

T2：行列均有序矩阵的查找与第 K 小

1 问题分析

- 给定 m×n 矩阵 A，满足：每行左到右升序、每列上到下升序。
- 任务 A：判断 target 是否存在，若存在可返回其坐标。
- 任务 B：将矩阵“概念上排序”后，返回第 K 小的值（考虑重复值）。

2 算法思路

A. 查找 target

思路 1

暴力破解法：遍历矩阵中的每一个元素，逐个与目标值 target 比较，若找到则返回存在，遍历结束仍未找到则返回不存在。（代码略）

```
bool findTarget(int** matrix, int rows, int cols, int target) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (matrix[i][j] == target) {
                return true; // 找到目标，返回 true
            }
        }
    }
    return false; // 遍历完未找到，返回 false
}
```

思路 2

从**右上角**出发：

- 设当前元素 x = A[i][j]
- 若 x == target：找到；
- 若 x > target：该列下方都更大 → j--（左移一列）；
- 若 x < target：该行右边都更大 → i++（下移一行）。
  这样每步都能**排除一整行或一整列**，直到越界或命中。

时间复杂度 O(m+n)，**空间** O(1)。

3 C 语言实现代码

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
bool searchTarget(int** arr, int m, int n,int target) {
    int i = 0; int j = n - 1;
    while (i<=n&&j>=0) {
        if (arr[i][j] == target)
            return true;
        else if (arr[i][j] > target)
            j--;
        else
            i++;
    }
    return false;
}
int main() {
    int m, n;
    printf("输入行数m与列数n\n");
    scanf("%d%d", &m, &n);
    int** arr = (int**)malloc(m * sizeof(int*));
    for (int i = 0; i < m; i++) {
        arr[i] = (int*)malloc(sizeof(int) * n);
    }
    printf("输入矩阵元素：\n");
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &arr[i][j]);
        }
    }
    int target;
    printf("taget:");
    scanf("%d", &target);
    if (searchTarget(arr,m,n,target))
        printf("找到了");
    else
        printf("没找到");
    for (int i = 0; i < m; i++) {
        free(arr[i]);
    }
    free(arr);
    return 0;
}
```

4 运行示例

```
输入行数m与列数n
4
4
输入矩阵元素：
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
taget:9
找到了
```

5 性能分析

方法	时间复杂度	空间复杂度	核心思路差异
暴力破解法	O(M*N)	O(1)	遍历矩阵所有元素，逐个比较，不利用矩阵的有序性。
阶梯式查找法	O(M +N)	O(1)	从右上角（或左下角）出发，利用 “行升序、列升序” 特性，每次排除一行或一列，快速缩小范围。

2 算法思路

B. 第 K 小

方案 B1：归并排序法（利用有序性进行优化）

- 利用矩阵 “每行升序” 特性：将 m 行视为 m 个已排序的子数组；
- 归并排序合并子数组：借鉴归并排序的 “合并阶段”，用额外空间维护合并后的有序数组；
- 取第 K 小数：合并过程中无需完全合并所有元素，合并到第 K 个元素时直接返回。

方案 B2：二分查找+阶梯计数法

- 确定二分范围：左边界 left= 矩阵最小值（左上角 matrix[0][0]），右边界 right= 矩阵最大值（右下角 matrix[m-1][n-1]）；
- 阶梯式统计元素：对中间值 mid，从矩阵右上角出发，统计≤mid 的元素个数 count（≤mid 则当前行左侧全符合，向下移；否则向左移）；

- 调整二分边界：若 count < k，说明第 K 小数在 mid 右侧（left=mid+1）；否则在左侧（right=mid）；
- 终止条件：left==right 时，即为第 K 小数。

3 C 语言代码实现 B1

```c
#include <stdio.h>
#include <stdlib.h>
int* mergeTwoArr(int* arr1, int size1, int* arr2, int size2) {
    int* merged = (int*)malloc((size1 + size2) * sizeof(int));
    int i = 0; int j = 0; int idx = 0;
    while (i < size1 && j < size2) {
        merged[idx++] = arr1[i] < arr2[j] ? arr1[i++] : arr2[j++];
    }
    while (i < size1)
        merged[idx++] = arr1[i++];
    while (j < size2)
        merged[idx++] = arr2[j++];
    return merged;
}//合并两数组
int* findKthsmall(int** arr, int m, int n, int k) {
    int* merged = (int*)malloc(n * sizeof(int));
    int mergedSize = n;
    for (int i = 0; i < n; i++)
        merged[i] = arr[0][i];//初始合并第一行
    for (int i = 1; i < m; i++) {//逐行合并
        int* temp = mergeTwoArr(merged, mergedSize, arr[i], n);
        free(merged);
        merged = temp;
        mergedSize += n;
        if (mergedSize > k) {//优化：若合并后数组元素>K,可提前截取
            int* tempK = (int*)malloc(k * sizeof(int));
            for (int j = 0; j < k; j++)
                tempK[j] = merged[j];
            free(merged);
            merged = tempK;
            mergedSize = k;
        }
    }
    int result = merged[k - 1];
    free(merged);
    return result;
}
```

```
38  int main() {
39      int m, n;
40      printf("输入行数m与列数n\n");
41      scanf("%d%d", &m, &n);
42      int** arr = (int**)malloc(m * sizeof(int*));
43      for (int i = 0; i < m; i++) {
44          arr[i] = (int*)malloc(sizeof(int) * n);
45      }
46      printf("输入矩阵元素：\n");
47      for (int i = 0; i < m; i++) {
48          for (int j = 0; j < n; j++) {
49              scanf("%d", &arr[i][j]);
50          }
51      }
52      int k;
53      printf("输入要查找的k(第K小)");
54      scanf("%d", &k);
55      printf("归并法求第K小元素为：%d, findKthsmall(arr, m, n, k));
56      for (int i = 0; i < m; i++)
57          free(arr[i]);
58      free(arr);
59      return 0;
60  }
```

运行示例

```
输入行数m与列数n
4
4
输入矩阵元素：
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
输入要查找的k(第K小)
7
归并法求第K小元素为： 7
```

C 语言代码实现 B2

```c
#include <stdio.h>
#include <stdlib.h>
int countLessOrEqual(int** arr, int m, int n, int mid) {
    int i = 0, j = n - 1, count = 0;
    while (i < m && j >= 0) {
        if (arr[i][j] <= mid) {
            count += j + 1; // 当前行左侧j+1个元素均符合
            i++; // 向下移一行
        }
        else {
            j--; // 向左移一列
        }
    }
    return count;
}
int findKthsmall(int** arr, int m, int n, int k) {
    int left = arr[0][0];
    int right = arr[m - 1][n - 1];
    while (left < right) {
        int mid = left + (right - left) / 2; // 避免溢出
        int cnt = countLessOrEqual(arr, m, n, mid);
        if (cnt < k) {
            left = mid + 1; // 第K小数在右侧
        }
        else {
            right = mid; // 第K小数在左侧（含mid）
        }
    }
    return left;
}
int main() {
    int m, n;
    printf("输入行数m与列数n\n");
    scanf("%d%d", &m, &n);
    int** arr = (int**)malloc(m * sizeof(int*));
    for (int i = 0; i < m; i++) {
        arr[i] = (int*)malloc(sizeof(int) * n);
    }
    printf("输入矩阵元素：\n");
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &arr[i][j]);
        }
    }
    int k;
    printf("输入要查找的k(第K小)");
    scanf("%d", &k);
    printf("二分法求第K小元素为：%d", findKthsmall(arr, m, n, k));
    for (int i = 0; i < m; i++)
        free(arr[i]);
    free(arr);
    return 0;
}
```

运行示例

```
输入行数m与列数n
4
4
输入矩阵元素：
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
输入要查找的k(第K小)
6
二分法求第K小元素为： 6
```

4 性能分析

算法类型	时间复杂度	空间复杂度
归并排序法	O(mnlog m)	O (k)（优化后）
二分查找 + 阶梯计数法	O ((m+n)*log D)（D 为最大最小差值）	O(1)